

Solución tecnológica para comercializadoras de vehículos

Rojas Serrano Holmer Mateo

Garcia Paez John Sebastian

Villamil Cardozo Geremi Santiago

Corporación Universitaria Minuto de Dios

Facultad de Ingeniería

Ingeniería de Sistemas

Programación Orientada a Objetos NRC 60 – 80841

Bogotá D.C, octubre de 2025

Tabla de Contenido

(Contextualización)	3
(Requerimientos)	5
(Lista de sustantivos y clases)	9
(Representación Grafica)	10
(Clases, atributos y Metodos)	11
(Documentación)	15
(Codificación inicial de clases).....	16
(Prototipos de interfaz).....	21
(Verificación, pruebas y cierre del proyecto).....	27
(Github).....	32

Fase 0:

Contextualización del enunciado del proyecto:

0.1 ¿Qué debemos hacer?

Diseñar y desarrollar una solución tecnológica orientada a objetos que apoye el proceso comercial de una empresa que vende vehículos.

El sistema debe permitir, como mínimo:

- Registrar prospectos y clientes.
- Gestionar el inventario de vehículos.
- Generar cotizaciones.
- Crear órdenes de venta.
- Ofrecer información clara para la toma de decisiones comerciales.

0.2 ¿Qué se necesita?

Se necesita un sistema de información que unifique la gestión de preventa y venta, de forma ordenada y confiable:

- Registro ordenado de prospectos, clientes e interacciones.
- Control de inventario de vehículos por vitrina o concesionario.
- Generación y seguimiento de cotizaciones, con su vigencia y estado.
- Creación de órdenes de venta, pagos y entregas de los vehículos.
- Reportes que muestren el resultado de campañas y el desempeño de ventas.

0.3 Elementos (objetos) que se identifican

Desde el enunciado se identifican elementos del mundo real que luego se convierten en clases del sistema:

- Empresa, red de concesionarios y vitrinas.

- Vehículos (marca, modelo, tipo, país de ensamble).
- Campañas de mercadeo e incentivos.
- Prospectos, clientes y asesores comerciales.
- Procesos de preventa, cotización, venta y servicio.

0.4 ¿Qué objetos se incluye?

Se incluyen los objetos que participan directamente en el proceso comercial y que aportan información al sistema:

- Prospecto, Cliente, Asesor.
- Campania, Interaccion, TestDrive.
- Concesionario, Vitrina, Inventario, Vehiculo.
- Cotizacion, OrdenDeVenta, Pago, Reporte.

0.5 ¿Cómo lo genera y qué objetos se requiere?

El sistema se genera a partir de:

- El análisis del caso de estudio.
- Las necesidades del negocio.
- Los requerimientos definidos junto con el docente.

Para lograrlo se trabaja con objetos que representan información (datos) y comportamiento (métodos) del proceso comercial. Estos objetos se relacionan entre sí para contar la historia completa: desde que un prospecto llega por una campaña hasta que sale conduciendo su vehículo.

Fase I:

Listado de requerimientos del contexto planteado. Cada requerimiento se documenta con la tabla: Nombre, Descripción, Entradas y Salidas.

Nombre: R-01 Interfaz externa y GUI (Grupo 1)

Descripción

Pantallas y navegación estandarizadas; distribución de elementos, resoluciones y mensajes de error coherentes para que el usuario se sienta orientado en todo momento.

Entradas

- Guía de estilo y prototipos de pantallas.
- Mapa de navegación del sistema.
- Resoluciones objetivo de los equipos donde se usará el sistema.

Salidas

- Vistas consistentes en todas las pantallas.
- Mensajes de error uniformes y comprensibles.
- Accesos directos y menús claramente definidos.

Nombre: R-02 Interfaces de hardware (Grupo 2)

Descripción

Compatibilidad con periféricos en vitrinas (impresoras, lectores QR/código, etc.), definiendo cómo se comunican con el sistema.

Entradas

- Inventario de periféricos soportados.
- Drivers y configuración de los dispositivos.
- Protocolos de comunicación utilizados.

Salidas

- Operaciones de impresión y lectura ejecutadas correctamente.

- Lista certificada de dispositivos que funcionan con el sistema.

Nombre: R-03 Interfaces de software (Grupo 3)

Descripción

Conexión con software externo: APIs (por ejemplo REST/JSON), bases de datos y sistemas heredados, para compartir información de forma segura.

Entradas

- Especificación de las APIs (por ejemplo, documentación tipo OpenAPI).
- Conectores y credenciales de acceso.
- Esquemas de datos que se comparten entre sistemas.

Salidas

- Endpoints integrados y funcionando.
- Importación y exportación de datos entre aplicaciones.
- Trazabilidad de las integraciones (se sabe qué se envía y qué se recibe).

Nombre: R-04 Desempeño y concurrencia (Grupo 4)

Descripción

Tiempos de respuesta definidos y número de usuarios/operaciones concurrentes medidos en escenarios típicos de uso.

Entradas

- Escenarios de carga definidos (por ejemplo: cuántos usuarios a la vez).
- Volúmenes estimados de datos y transacciones.
- Herramientas o scripts de prueba de desempeño.

Salidas

- Informe de tiempos de respuesta menores o iguales al objetivo.
- Capacidad concurrente validada para el contexto del proyecto.

Tolerancia a fallos (safety) (Grupo 5)

Descripción

Prevención de acciones peligrosas y definición de una política de respaldo y recuperación ante pérdida o daño de información.

Entradas

- Matriz de riesgos y listado de acciones críticas (por ejemplo, borrado masivo).
- Plan de respaldo (backup) y restauración de datos.

Salidas

- Confirmaciones y registros de auditoría de las acciones críticas.
- Respaldos realizados y restauración verificada en pruebas.

Nombre: R-06 Seguridad y calidad (usuario) (Grupo 6)

Descripción

Autenticación y autorización de usuarios, protección de la información, disponibilidad del sistema y uso eficiente de recursos.

Entradas

- Matriz de roles y permisos.
- Políticas de seguridad (contraseñas, manejo de sesiones, etc.).
- Monitoreo de disponibilidad (tiempo que el sistema está “arriba”).

Salidas

- Control de acceso aplicado según rol.
- Métricas de disponibilidad del sistema.
- Registros de auditoría sobre quién hizo qué y cuándo.

Nombre: R-07 Interoperabilidad, confiabilidad, robustez, usabilidad, instalación (Grupo 7)

Descripción

El sistema debe interoperar con su entorno, ser confiable y usable para las personas, y contar con procesos de instalación claros y repetibles.

Entradas

- Checklist de interoperabilidad y robustez.
- Guías de instalación y de uso.
- Pruebas con usuarios (pruebas de usabilidad).

Salidas

- Instalación validada en el entorno objetivo.
- Encuestas o resultados de usabilidad aceptables.
- Estabilidad operativa en las pruebas (el sistema no se “cae” fácilmente).

Nombre: R-08 Mantenibilidad y documentación (desarrollador) (Grupo 8)

Descripción

Estructura de directorios y metodología de diseño definidas, además de estándares de documentación y de código para facilitar el mantenimiento.

Entradas

- Guía de arquitectura del sistema.
- Convenciones del proyecto (nombres, paquetes, estilo de código).
- Plantillas de documentación técnica.

Salidas

- Documentación técnica disponible y actualizada.
- Estructura modular y legible del código fuente.

Nombre: R-09 Portabilidad, reusabilidad y pruebas (desarrollador) (Grupo 9)

Descripción

Ejecución del sistema en las plataformas objetivo, desarrollo de componentes reutilizables y facilidades para realizar pruebas.

Entradas

- Matriz de plataformas objetivo (por ejemplo: Windows, Linux, laboratorio).
- Estrategia de pruebas y datos semilla.

Salidas

- Builds o versiones ejecutables en las distintas plataformas.
- Librerías o componentes reutilizables dentro del proyecto.
- Suites de prueba (manuales o automatizadas) preparadas para validar el sistema.

Fase II. Diseño de clases (modelo orientado a objetos)

En esta fase tomamos el caso de estudio de la comercializadora de vehículos y lo transformamos en un modelo orientado a objetos. A partir del texto se identifican sustantivos, se proponen clases, se describen sus atributos y métodos y se deja una base lista para pasar a la codificación en Java.

1. Lista de sustantivos del enunciado y clases candidatas

A partir del caso de estudio se identifican los siguientes sustantivos relevantes:

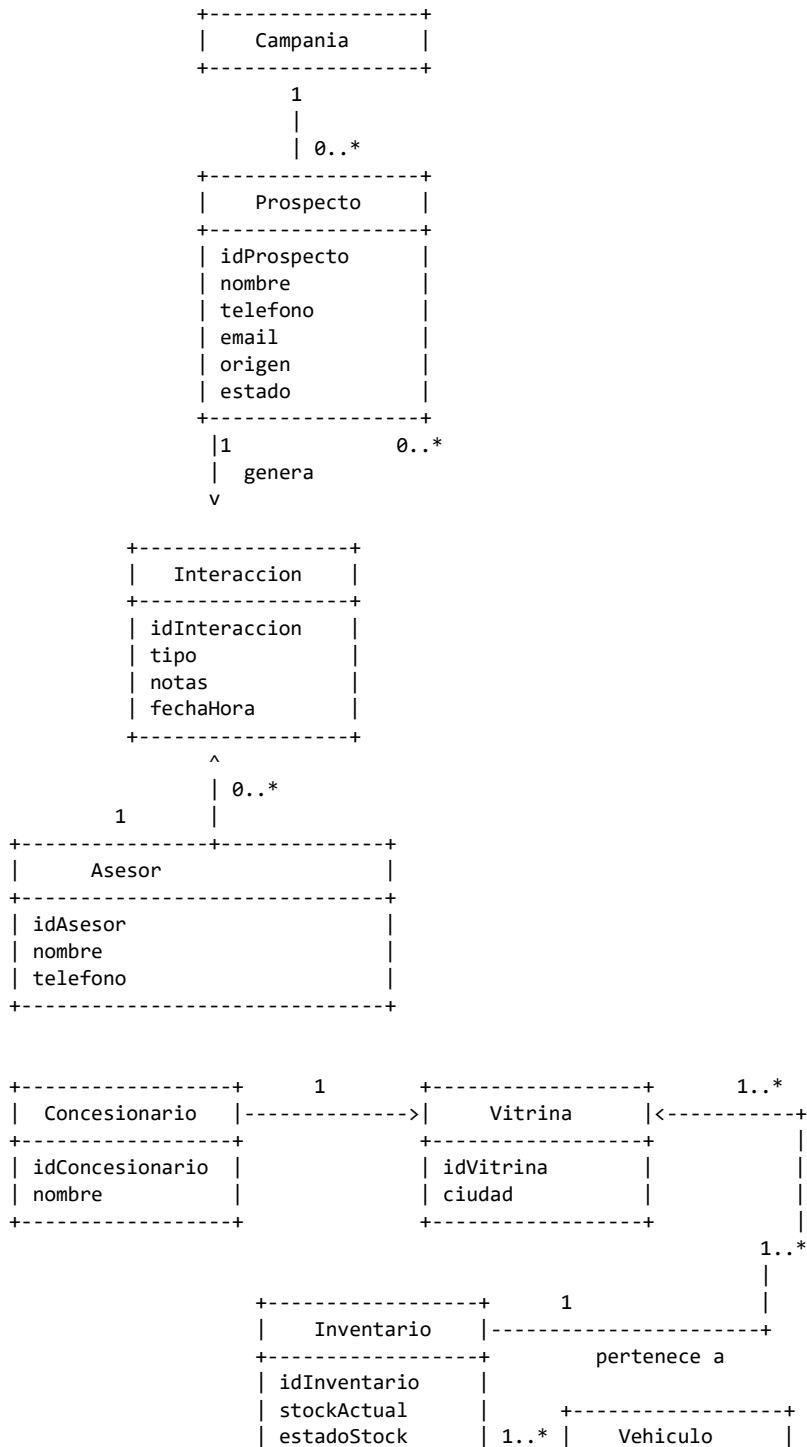
Empresa, concesionario, vitrina, red de distribución, vehículo, marca, tipo de vehículo, país de ensamble, campaña, publicidad, mercadeo, incentivo, cliente potencial (prospecto), cliente, proceso de compra, servicio, tiempo de decisión, asesor, atención, seguimiento, reporte, indicador, ventas, preventa, sistema de información.

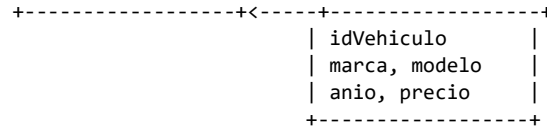
De estos sustantivos se seleccionan como clases principales del sistema:

Prospecto, Interaccion, Asesor, Campania, Vehiculo, Vitrina, Concesionario, Inventario, Cotizacion, OrdenDeVenta, Cliente, TestDrive, Usuario, Rol, Reporte.

2. Representación gráfica simplificada de clases y relaciones

A continuación se muestra una representación simplificada de las relaciones principales entre las clases del sistema. Esta vista sirve como guía para el diagrama UML de clases que se consigna en la plantilla.





Prospecto 1..* ---- 0..* Cotizacion ---- 1 Vehiculo
 Cotizacion 1 ---- 0..1 OrdenDeVenta
 OrdenDeVenta 1 ---- 1 Vehiculo (reservado y luego vendido)

3. Clases principales: atributos y métodos

En esta sección se describen las clases seleccionadas para el modelo, junto con sus atributos más importantes y las operaciones (métodos) asociadas.

3.1 Clase: Prospecto

Atributos:

- idProspecto: String – identificador único del prospecto.
- nombre: String – nombre completo.
- telefono: String – número de contacto.
- email: String – correo electrónico.
- origen: String – campaña, vitrina o referencia.
- estado: String – nuevo, en_seguimiento, ganado, perdido.
- fechaRegistro: DateTime – fecha y hora de creación del registro.

Métodos:

- registrarInteraccion(tipo, notas, fechaHora): agrega una interacción al historial.
- cambiarEstado(nuevoEstado): actualiza el estado del prospecto en el proceso de venta.
- asignarAsesor(asesor): vincula el prospecto con un asesor responsable.

3.2 Clase: Interaccion

Atributos:

- idInteraccion: String – identificador único.
- prospecto: Prospecto – referencia al prospecto relacionado.
- asesor: Asesor – asesor que realiza la interacción.
- tipo: String – llamada, visita, correo, mensaje, etc.
- notas: String – detalle de lo tratado.
- fechaHora: DateTime – momento en que ocurre la interacción.

Métodos:

- resumen(): devuelve una descripción corta de la interacción.

3.3 Clase: Asesor

Atributos:

- idAsesor: String – identificador del asesor.
- nombre: String – nombre completo.
- telefono: String – contacto principal.
- email: String – correo institucional.
- vitrina: Vitrina – vitrina donde trabaja el asesor.

Métodos:

- asignarProspecto(prospecto): asocia un prospecto al asesor.
- generarCotizacion(prospecto, vehiculo): crea una cotización para un prospecto.

3.4 Clase: Campania

Atributos:

- idCampania: String – identificador de la campaña.
- nombre: String – nombre comercial.
- tipo: String – radio, redes sociales, valla, etc.
- fechaInicio: Date – inicio de la campaña.
- fechaFin: Date – fin de la campaña.
- presupuesto: Decimal – monto asignado.

Métodos:

- calcularProspectosGenerados(): cuenta los prospectos asociados a esta campaña.
- calcularConversion(): calcula el porcentaje de prospectos que se convierten en venta.

3.5 Clase: Vehiculo

Atributos:

- idVehiculo: String – identificador interno.

- vin: String – número único de chasis.
- marca: String – marca del fabricante.
- modelo: String – modelo del vehículo.
- anio: Int – año modelo.
- precioLista: Decimal – precio base de venta.
- estado: String – disponible, reservado o vendido.

Métodos:

- esDisponible(): indica si el vehículo está disponible para ser vendido.
- cambiarEstado(nuevoEstado): actualiza el estado del vehículo.

3.6 Clase: Vitrina

Atributos:

- idVitrina: String – identificador de la vitrina.
- nombre: String – nombre comercial.
- ciudad: String – ciudad donde se encuentra.
- direccion: String – dirección física.
- concesionario: Concesionario – concesionario al que pertenece.

Métodos:

- obtenerStock(): consulta el inventario asociado a la vitrina.
- contarProspectosActivos(): devuelve el número de prospectos en seguimiento.

3.7 Clase: Inventario

Atributos:

- idInventario: String – identificador del registro de inventario.
- vitrina: Vitrina – vitrina responsable.
- vehiculo: Vehiculo – vehículo al que corresponde el registro.
- estadoStock: String – disponible, reservado o vendido.
- fechaActualizacion: DateTime – fecha de última actualización.

Métodos:

- reservar(ordenVenta): marca el vehículo como reservado para una orden de venta.
- liberar(): devuelve el vehículo a disponible si la venta no se concreta.

3.8 Clase: Cotizacion

Atributos:

- idCotizacion: String – identificador de la cotización.
- prospecto: Prospecto – destinatario de la cotización.
- vehiculo: Vehiculo – vehículo ofertado.
- asesor: Asesor – asesor que la genera.
- precioFinal: Decimal – precio después de descuentos.
- descuento: Decimal – valor o porcentaje de descuento.
- vigencia: Date – fecha límite de validez.
- estado: String – abierta, aceptada o vencida.

Métodos:

- calcularPrecioFinal(): calcula el precio final aplicando el descuento.
- esVigente(): indica si la cotización no ha vencido.
- marcarComoAceptada(): cambia el estado y permite generar la orden de venta.

3.9 Clase: OrdenDeVenta

Atributos:

- idOrden: String – identificador de la orden.
- cotizacion: Cotizacion – cotización origen.
- vehiculo: Vehiculo – vehículo asociado.
- prospecto: Prospecto – comprador (o cliente).
- estado: String – creada, pagada o entregada.
- fechaCreacion: DateTime – momento en que se genera la orden.
- fechaEntrega: DateTime – fecha programada de entrega.

Métodos:

- registrarPago(monto): registra un pago asociado a la orden.
- programarEntrega(fecha): define la fecha de entrega del vehículo.
- cerrarOrden(): marca la orden como entregada y el vehículo como vendido.

4. Documentación de atributos de la clase

La siguiente tabla documenta algunos atributos clave de las clases, incluyendo su nombre de codificación y objetivo dentro del sistema.

Nombre de la clase	Nombre del atributo	Nombre Codificación	Objetivo
Prospecto	fechaRegistro	fecha_registro	Controlar cuándo se crea el prospecto y apoyar procesos de auditoría.
Prospecto	estado	estado	Identificar en qué etapa del proceso comercial se encuentra el prospecto.
Interaccion	tipo	tipo	Clasificar la interacción (llamada, visita, mensaje, correo, etc.).
Interaccion	fechaHora	fecha_hora	Ordenar cronológicamente el historial de interacciones.
Vehiculo	vin	vin	Identificar de forma única el chasis del vehículo.
Vehiculo	precioLista	precio_lista	Servir como base para calcular cotizaciones y descuentos.
Inventario	estadoStock	estado_stock	Indicar si el vehículo está disponible, reservado o vendido.
Cotizacion	vigencia	vigencia	Definir hasta cuándo es válida la cotización.
Cotizacion	estado	estado	Saber si la cotización está

			abierta, aceptada o vencida.
OrdenDeVenta	estado	estado	Controlar el avance de la orden hasta su entrega final.

Fase III. Codificación inicial de clases (diseño de métodos)

En esta fase no se programa todavía, sino que se documentan los métodos principales del sistema, a partir del modelo de clases definido en la Fase II.

La idea es dejar claro:

- Qué hace cada método.
- Qué datos necesita de entrada.
- Qué entrega como resultado.
- Cómo piensa resolver el problema.
- Cómo se verá su firma cuando se implemente en el código.

A continuación, se describen algunos métodos clave del sistema de la comercializadora de vehículos.

Método 1

Nombre del método: registrarInteraccion

Clase: Prospecto

Entrada:

- tipo: tipo de interacción (llamada, visita, mensaje, correo, etc.).
- notas: texto con el detalle de lo hablado.
- fechaHora: fecha y hora en que ocurrió la interacción.

Resultado:

- No devuelve un valor directo (se considera resultado void), pero el historial de interacciones del prospecto queda actualizado.

Solución planteada (explicación):

Cuando el asesor se comunica con un prospecto, se debe dejar evidencia.

El método registrarInteraccion creará un nuevo objeto de tipo Interaccion, lo asociará al Prospecto y lo agregará a una lista de interacciones. De esta forma, el sistema conserva el historial cronológico de contactos y permite hacer seguimiento al proceso de venta.

Firma (para cuando se programe):
 registrarInteraccion (tipo, notas, fechaHora)

Método 2

Nombre del método: cambiarEstado
 Clase: Prospecto

Entrada:

- nuevoEstado: estado al que se quiere mover el prospecto (por ejemplo: "nuevo", "en_seguimiento", "ganado", "perdido").

Resultado:

- No devuelve un valor (void), pero se actualiza el campo estado del prospecto.

Solución planteada:

Este método permitirá cambiar el estado del prospecto dentro del proceso comercial. Antes de actualizar, el método validará que el nuevo estado sea uno de los valores permitidos. Si es válido, actualiza el atributo estado. Con esto se sabrá si el prospecto está en seguimiento, si se perdió la oportunidad o si se convirtió en un cliente (ganado).

Firma (para cuando se programe):
 cambiarEstado (nuevoEstado)

Método 3

Nombre del método: generarCotizacion
 Clase: Asesor

Entrada:

- prospecto: el prospecto al que se le va a ofrecer la cotización.
- vehiculo: el vehículo específico que se quiere cotizar.

Resultado:

- Devuelve un objeto de tipo Cotizacion.

Solución planteada:

El asesor, desde su pantalla, selecciona un prospecto y un vehículo disponible. El método generarCotizacion creará una nueva Cotizacion, asociando el prospecto, el

vehículo y el asesor. También establecerá un estado inicial (por ejemplo, "abierta") y una vigencia. El resultado es una cotización lista para ser presentada al prospecto y, si la acepta, usada luego para generar la orden de venta.

Firma (para cuando se programe):
 generarCotizacion (prospecto, vehiculo)

Método 4

Nombre del método: calcularPrecioFinal
 Clase: Cotizacion

Entrada:

- No recibe parámetros externos. Utiliza internamente:
 - El precioLista del vehículo.
 - El descuento definido en la cotización.

Resultado:

- Devuelve el precio final que se le ofrecerá al prospecto (un valor numérico).

Solución planteada:

El método tomará el precio de lista del vehículo y le restará el descuento asignado en la cotización. El valor resultante se considera el precio final que el asesor mostrará al prospecto.

Este cálculo dejará almacenado el resultado en el atributo precioFinal de la cotización, y además lo devolverá para mostrarlo en pantalla o en un documento.

Firma (para cuando se programe):
 calcularPrecioFinal()

Método 5

Nombre del método: esVigente
 Clase: Cotizacion

Entrada:

- No recibe parámetros. Utiliza:
 - La fecha actual del sistema.
 - El atributo vigencia de la propia cotización.

Resultado:

- Devuelve un valor booleano:
 - true si la cotización todavía es válida.
 - false si la cotización ya está vencida.

Solución planteada:

El método compara la fecha actual con la fecha de vigencia de la cotización.

Si la fecha actual es menor o igual a la vigencia, la cotización sigue activa y se considera válida. Si ya pasó la fecha de vigencia, se considera vencida y el método devolverá false. Esto permite evitar que se acepten cotizaciones fuera de fecha.

Firma (para cuando se programe):
esVigente()

Método 6

Nombre del método: reservar
Clase: Inventario

Entrada:

- ordenVenta: referencia a la OrdenDeVenta para la cual se está reservando el vehículo.

Resultado:

- No devuelve un valor (void), pero:
 - Cambia el estadoStock del inventario (por ejemplo, de "disponible" a "reservado").
 - Cambia también el estado del vehículo a reservado.

Solución planteada:

Cuando se genera una orden de venta, se debe bloquear el vehículo para que no pueda ser vendido a otra persona.

El método reservar verificará que el vehículo esté disponible. Si lo está, actualizará el estado del registro de Inventario a "reservado" y también cambiará el estado del Vehículo. De esta manera se garantiza que el mismo carro no se venda dos veces.

Firma (para cuando se programe):
reservar (ordenVenta)

Método 7

Nombre del método: cerrarOrden

Clase: OrdenDeVenta

Entrada:

- inventarioAsociado: registro de Inventario que corresponde al vehículo que se va a entregar.

Resultado:

- No devuelve un valor (void), pero:
 - Cambia el estado de la orden a "entregada".
 - Cambia el estado del vehículo a "vendido" a través del inventario.

Solución planteada:

El método se utiliza al final del proceso comercial, cuando el cliente ya ha pagado y se le va a entregar el vehículo.

Primero se verifica que la orden se encuentre en estado "pagada". Si la condición se cumple, la orden cambia a "entregada" y se llama a un método del Inventario para marcar el vehículo como "vendido". Esto mantiene la consistencia de la información y cierra formalmente la venta.

Firma (para cuando se programe):
cerrarOrden(inventarioAsociado)

Fase IV. Prototipos de interfaz e implementación de la lógica

En esta fase se define cómo se verá el sistema para el usuario (pantallas de entrada y salida de datos) y se describe la lógica interna de algunos métodos clave, usando condicionales, ciclos y cálculos sencillos.

La idea es conectar lo que ya se analizó (Fases 0, I y II) con la forma en que el usuario realmente trabaja con el sistema.

IV.1 Prototipos de entrada (pantallas de captura de datos)

En esta sección se describen los prototipos de entrada, es decir, las pantallas donde el usuario digita o selecciona información.

IV.1.1 Pantalla “Registro de Prospecto”

Objetivo de la pantalla:

Permitir que el asesor o usuario registre un nuevo prospecto en el sistema con sus datos básicos de contacto y el origen de la oportunidad.

Elementos principales (entrada):

- Campo de texto: *ID prospecto* (puede ser generado automáticamente).
- Campo de texto: *Nombre completo*.
- Campo de texto: *Teléfono de contacto*.
- Campo de texto: *Correo electrónico*.
- Lista desplegable: *Origen* (campana, vitrina, referencia, otros).
- Botones:
 - Guardar (almacena el prospecto).
 - Limpiar (borra los datos del formulario).
 - Cancelar / Volver (retorna a la pantalla principal).

Validaciones básicas:

- El nombre no puede estar vacío.
- El teléfono y correo deben tener un formato válido.
- Debe seleccionarse al menos un origen.

IV.1.2 Pantalla “Registro de Interacción”

Objetivo:

Permitir registrar llamadas, visitas, mensajes u otros contactos realizados con un prospecto.

Elementos de entrada:

- Campo: *ID prospecto* (seleccionado desde una lista o buscador).
- Campo: *Asesor* (ya asignado o seleccionado).
- Lista desplegable: *Tipo de interacción* (llamada, visita, mensaje, correo, etc.).
- Campo de texto grande: *Notas / Detalle de la interacción*.
- Campo de fecha y hora: *Fecha y hora de la interacción* (por defecto, la actual).
- Botones: Guardar, Cancelar.

Validaciones:

- Debe existir un prospecto seleccionado.
- El tipo de interacción es obligatorio.
- Las notas no deben estar totalmente vacías (mínimo algunas palabras).

IV.1.3 Pantalla “Generación de Cotización”

Objetivo:

Permitir que el asesor genere una cotización para un prospecto, seleccionando un vehículo disponible del inventario.

Elementos de entrada:

- Campo/buscador: *Prospecto* (por nombre o ID).
- Lista o tabla: *Vehículos disponibles* (marca, modelo, año, precio lista, estado).
- Campo numérico: *Descuento* (valor o porcentaje).
- Campo de fecha: *Vigencia de la cotización*.
- Botones:
 - Calcular precio final.
 - Guardar cotización.
 - Cancelar.

Validaciones:

- El vehículo debe estar en estado “disponible”.
- El descuento no puede dejar el precio final en negativo.
- La fecha de vigencia debe ser mayor o igual a la fecha actual.

IV.2 Prototipos de salida (pantallas de consulta y reportes)

Ahora se describen las pantallas de salida, es decir, vistas de consulta, listados o reportes que el sistema genera.

IV.2.1 Pantalla “Listado de Prospectos”

Objetivo:

Mostrar una lista de prospectos registrados, con la posibilidad de filtrarlos y ver su estado.

Elementos de salida:

- Tabla con columnas:
 - ID prospecto.
 - Nombre.
 - Teléfono.
 - Estado (nuevo, en seguimiento, ganado, perdido).
 - Asesor asignado.
 - Fecha de registro.
- Filtros:
 - Por estado.
 - Por asesor.
 - Por rango de fechas.

Acciones posibles:

- Ver detalle del prospecto.
- Acceder al historial de interacciones.
- Cambiar estado (por ejemplo, marcar como ganado o perdido).

IV.2.2 Pantalla “Detalle de Cotización”

Objetivo:

Presentar los datos de una cotización específica generada para un prospecto.

Elementos de salida:

- Datos del prospecto (nombre, contacto).
- Datos del vehículo (marca, modelo, año, precio lista).
- Campo “Descuento aplicado”.

- Campo “Precio final”.
- Estado de la cotización (abierta, aceptada, vencida).
- Vigencia (fecha límite).

Acciones:

- Botón “Aceptar cotización” (si está vigente).
- Botón “Generar orden de venta” (si está aceptada).

IV.2.3 Pantalla “Reporte de Ventas”

Objetivo:

Permitir visualizar, de forma resumida, el resultado de las ventas en un período de tiempo.

Elementos de salida:

- Filtros de entrada:
 - Rango de fechas.
 - Vitrina.
 - Asesor.
- Elementos mostrados:
 - Número de órdenes de venta cerradas (entregadas).
 - Monto total de ventas.
 - Lista de vehículos vendidos (ID, marca, modelo, precio).
 - Posible resumen por campaña o por asesor.

IV.3 Lógica interna de algunos métodos (condicionales, ciclos y cálculos)

En esta sección se describe, a nivel lógico (no en código Java real), cómo funciona internamente la lógica de algunos métodos. Esta descripción sirve de puente entre la Fase III y la implementación final.

IV.3.1 Lógica del método calcularPrecioFinal (Cotizacion)

Objetivo:

Calcular el precio final de una cotización aplicando el descuento al precio de lista del vehículo.

Lógica (en palabras):

1. Tomar el precio de lista del vehículo asociado a la cotización.
2. Tomar el descuento definido para esta cotización.
3. Calcular:

$$\text{precio_final} = \text{precio_lista} - \text{descuento}$$

4. Guardar el resultado en el atributo precioFinal.
5. Devolver ese valor para mostrarlo en la interfaz.

Condicionales / Validaciones posibles:

- Si el descuento es mayor al precio de lista, avisar que el descuento no es válido.

IV.3.2 Lógica del método esVigente (Cotizacion)

Objetivo:

Verificar si una cotización sigue siendo válida en función de su fecha de vigencia.

Lógica resumida:

1. Obtener la fecha actual del sistema.
2. Compararla con la fecha de vigencia de la cotización.
3. Si la fecha actual es menor o igual a la vigencia → la cotización es vigente.
4. Si la fecha actual es mayor a la vigencia → la cotización está vencida.

Uso de condicional:

- Si (fecha_actual <= vigencia) → devolver verdadero.
- En caso contrario → devolver falso.

IV.3.3 Lógica del método reservar (Inventario)

Objetivo:

Marcar un vehículo como “reservado” para una orden de venta, evitando que sea vendido a otra persona.

Pasos lógicos:

1. Verificar si el vehículo asociado al registro de inventario está disponible.
2. Si el vehículo NO está disponible, detener la operación y mostrar un mensaje de error.
3. Si el vehículo está disponible:
 - Cambiar el estado del inventario a “reservado”.

- Cambiar el estado del vehículo a “reservado”.
- Actualizar la fecha de última modificación del inventario.

Condicional:

- Si estado del vehículo $\neq$ "disponible" $\rightarrow$ no permitir la reserva.
- Si sí es "disponible" $\rightarrow$ continuar con la reserva.

IV.3.4 Lógica del método cerrarOrden (OrdenDeVenta)

Objetivo:

Cerrar una orden de venta cuando la venta ya se pagó y el vehículo está listo para entregar.

Pasos lógicos:

1. Verificar que la orden se encuentre en estado “pagada”.
2. Si la orden NO está pagada, no se permite cerrarla y se muestra un mensaje de advertencia.
3. Si la orden está pagada:
 - Cambiar el estado de la orden a “entregada”.
 - Llamar a la lógica de Inventario para marcar el vehículo como “vendido”.
 - Registrar la fecha de entrega.

Condicional:

- Si estado $\neq$ "pagada" $\rightarrow$ no se puede cerrar la orden.
- Si estado = "pagada" $\rightarrow$ se cambia a "entregada" y se actualiza el inventario.

IV.3.5 Lógica del flujo completo (resumen narrado)

Como síntesis de la Fase IV, la lógica del sistema conecta las pantallas y métodos así:

1. Se registra un prospecto (pantalla de entrada).
2. Se registran interacciones que alimentan su historial.
3. El asesor genera una cotización seleccionando un vehículo disponible.
4. Se calcula el precio final aplicando descuentos.
5. Se verifica si la cotización sigue vigente antes de aceptarla.
6. Si se acepta, se crea una orden de venta.
7. El inventario reserva el vehículo durante el proceso de pago.
8. Cuando se completa el pago, se cierra la orden y el vehículo se marca como vendido.
9. Finalmente, todo esto se refleja en las pantallas de salida y reportes.

Fase V. Verificación, pruebas y cierre del proyecto

En esta fase se verifica que el sistema cumpla con los requerimientos definidos en la Fase I y con el diseño realizado en las Fases II, III y IV.

También se dejan evidencias del funcionamiento (pruebas y pantallazos) y se hace una reflexión final sobre el trabajo realizado.

La intención es que no sea solo “un código que compila”, sino un sistema que tiene sentido frente al problema de la comercializadora de vehículos.

V.1 Verificación frente a los requerimientos

Aquí se revisan los 9 grupos de requerimientos definidos en la Fase I y se verifica, de manera cualitativa, cómo el diseño y la futura implementación los atienden.

Puedes llevar esto a una tabla con columnas:

Requerimiento – Descripción – Cómo se verifica / evidencia.

R-01 Interfaz externa y GUI (Grupo 1)

- Descripción: Pantallas y navegación estandarizadas; distribución, resoluciones y mensajes de error coherentes.
- Verificación / Evidencia:
 - Prototipos de pantallas definidos en la Fase IV (registro de prospectos, registro de interacciones, generación de cotizaciones, listados).
 - Mensajes claros para validaciones (datos obligatorios, formatos incorrectos).
 - Flujo de navegación consistente entre las pantallas principales.

R-02 Interfaces de hardware (Grupo 2)

- Descripción: Compatibilidad con periféricos en vitrinas (por ejemplo, impresora para cotizaciones y órdenes).
- Verificación / Evidencia:
 - Para el alcance académico, se verifica que el sistema pueda generar documentos (cotizaciones / órdenes) que se puedan imprimir desde el equipo.
 - Se deja documentado que el sistema está preparado para integrarse con dispositivos de impresión del laboratorio o la empresa.

R-03 Interfaces de software (Grupo 3)

- Descripción: Conexión con software externo, especialmente base de datos y sistema operativo.
- Verificación / Evidencia:
 - El diseño de clases está preparado para trabajar con persistencia (clases modelo limpias, sin depender de la interfaz).
 - Se documenta que la información puede guardarse en una base de datos o en archivos, según el entorno donde se implemente.

R-04 Desempeño y concurrencia (Grupo 4)

- Descripción: Tiempos de respuesta aceptables y manejo básico de varios usuarios.
- Verificación / Evidencia:
 - Para el contexto de laboratorio, se verifican tiempos de respuesta visualmente aceptables al registrar prospectos, cotizaciones y órdenes.
 - Pruebas con más de una operación seguida (ej.: varios registros de prospectos y cotizaciones) sin que la aplicación se bloquee.

R-05 Tolerancia a fallos (safety) (Grupo 5)

- Descripción: Prevención de acciones peligrosas y respaldo básico de la información.
- Verificación / Evidencia:
 - Mensajes de confirmación para acciones sensibles (ej.: borrar un registro).
 - Manejo básico de errores (no permitir continuar con datos obligatorios vacíos).
 - Propuesta de respaldo simple: guardar la información en archivos o BD para que, si el programa se cierra, no se pierdan los datos ya registrados.

R-06 Seguridad y calidad (usuario) (Grupo 6)

- Descripción: Control de acceso básico, protección de la información y disponibilidad.
- Verificación / Evidencia:
 - Diseño preparado para tener usuarios con rol “asesor” o “administrador”.
 - Posibilidad de restringir acceso a ciertas funciones (ej.: solo el administrador puede ver reportes globales).

- Se deja documentada la idea de una pantalla de inicio de sesión (login) como mejora.

R-07 Interoperabilidad, confiabilidad, robustez, usabilidad, instalación (Grupo 7)

- Descripción: El sistema debe ser usable, estable y tener una instalación clara.
- Verificación / Evidencia:
 - Prototipos centrados en la simplicidad y en un flujo lógico: primero prospectos, luego interacciones, cotizaciones, órdenes.
 - Pruebas funcionales donde el sistema se ejecuta sin fallos críticos durante los escenarios de uso más comunes.
 - Pasos de instalación documentados (por ejemplo: tener Java, importar proyecto, ejecutar clase principal).

R-08 Mantenibilidad y documentación (Grupo 8)

- Descripción: Código organizado y documentación disponible.
- Verificación / Evidencia:
 - Clases separadas por responsabilidad (modelo, servicio, etc.).
 - Documentación del diseño en este documento (Fases II, III y IV).
 - Comentarios en el código (cuando se implemente) siguiendo el estilo acordado.

R-09 Portabilidad, reusabilidad y pruebas (Grupo 9)

- Descripción: Posibilidad de ejecutar el sistema en diferentes entornos académicos y facilidad para hacer pruebas.
- Verificación / Evidencia:
 - Uso de Java, que permite portar el proyecto entre equipos de laboratorio.
 - Clases y métodos pensados para ser reutilizables en otros módulos o futuros proyectos.
 - Pruebas básicas documentadas del flujo principal: del prospecto a la orden de venta.

V.2 Plan y evidencias de prueba

En esta sección se describen los escenarios de prueba (casos de uso probados) y las evidencias que se podrían adjuntar (pantallazos, resultados, etc.).

Puedes llevar esto a una tabla con:

Caso de prueba – Descripción – Datos de entrada – Resultado esperado – Resultado obtenido.

Caso de prueba 1: Registro de prospecto

- Descripción: Verificar que el sistema permita registrar un nuevo prospecto con sus datos básicos.
- Datos de entrada:
 - Nombre: “Juan Pérez”.
 - Teléfono: “3001234567”.
 - Email: “juan@example.com”.
 - Origen: “Campaña redes sociales”.
- Resultado esperado:
 - El prospecto queda guardado con estado “nuevo”.
 - Aparece en el listado de prospectos.
- Resultado obtenido:
 - (Aquí, cuando se implemente, se indica si se logró y se puede adjuntar un pantallazo).

Caso de prueba 2: Registro de interacción

- Descripción: Verificar que se pueda registrar una llamada o visita asociada a un prospecto.
- Datos de entrada:
 - Prospecto: “Juan Pérez”.
 - Tipo: “llamada”.
 - Notas: “Se informó sobre el plan de financiación”.
 - Fecha y hora: fecha actual.
- Resultado esperado:
 - La interacción aparece en el historial de ese prospecto.

Caso de prueba 3: Generación de cotización

- Descripción: Generar una cotización para un prospecto con un vehículo disponible.
- Datos de entrada:
 - Prospecto: “Juan Pérez”.
 - Vehículo: “Sedán X 2024, \$80.000.000”.
 - Descuento: “\$5.000.000”.
- Resultado esperado:
 - Se crea una cotización en estado “abierta”.

- El precio final se muestra como \$75.000.000.

Caso de prueba 4: Aceptación de cotización y creación de orden de venta

- Descripción: Aceptar una cotización vigente y generar una orden de venta.
- Datos de entrada:
 - Cotización en estado “abierta” y con fecha de vigencia válida.
- Resultado esperado:
 - La cotización pasa a estado “aceptada”.
 - Se crea una orden de venta asociada a esa cotización.

Caso de prueba 5: Cierre de orden de venta

- Descripción: Cerrar una orden de venta una vez que se haya registrado el pago.
- Datos de entrada:
 - Orden de venta en estado “pagada”.
- Resultado esperado:
 - La orden pasa a estado “entregada”.
 - El vehículo se marca como “vendido” en el inventario.

V.3 Reflexión y cierre del proyecto

Finalmente, se realiza una reflexión sobre el desarrollo del proyecto:

- A partir de un caso de estudio textual (la comercializadora de vehículos), se logró identificar objetos, procesos y necesidades reales.
- Se construyó un modelo orientado a objetos con clases, atributos y métodos que cuentan la historia completa: desde el primer contacto con un prospecto hasta la entrega del vehículo.
- Se documentaron requerimientos, diseño, métodos y lógica de negocio, pensando tanto en el usuario final como en futuros desarrolladores.
- La codificación que se realizará en Java se apoya en todas estas fases, lo que permite que el sistema no nazca “a ciegas”, sino con un sentido claro y humano.

Este proyecto no solo deja una aplicación, sino también un aprendizaje: que detrás de cada pantalla y cada línea de código hay personas, procesos y decisiones que intentamos acompañar con una solución tecnológica sencilla, organizada y hecha con cuidado.

Link de Github:

<https://github.com/SebastianGarcia0815/poo-comercializadora-vehiculos-g10>